



Module Instructor:	Obinna Izima
Assessment Title:	Cloud Incident Monitoring and Response Platform
Student ID	20098086
Student Name	Mahendra Shabade
Module Title:	Web Development for Information Systems
Course	MSc Information System
Summery word count	244
Main Text Work Count	1909
Total Word Count	2366
GitHub Link	https://github.com/MShabade/cloud-incident-monitoring-system

Table of Contents

1. Introduction.....	1
1.1 Background and Research	1
1.2 Report Structure.....	2

2. Project Overview	2
2.1 Problem Statement.....	2
2.2 Full-Stack Objectives	3
3. System Requirements.....	3
3.1 Functional Requirements.....	3
3.2 Non-Functional Requirements.....	4
4. System Architecture.....	4
4.1 Client-Server Design	4
4.2 Folder Structure	5
5. Data Model.....	5
5.1 Incident Document	5
5.2 User Document.....	6
6. CRUD Operations	6
6.1 Create.....	6
6.2 Read.....	7
6.3 Update.....	7
6.4 Delete.....	8
7. Additional Features	8
8. Testing and Validation.....	9
9. External API Integration	10
10. Conclusion	11
Reflective Learning Experience.....	11
References.....	12

1. Introduction

1.1 Background and Research

Most web projects today are split into a frontend, an API, a database, and a hosting layer rather than one single program. For B9IS109 I set out to build a complete information system, not a small isolated lab. I chose cloud incident monitoring because it covers all of these parts in a way that feels close to real platform work.

Before writing much code I reviewed how REST supports client-server design ([Fielding, 2000](#)), how JWT tokens allow stateless login ([JSON Web Token Introduction, 2024](#)), and how MongoDB stores flexible documents compared with fixed SQL tables ([MongoDB Inc., 2025](#); [Mongoose, 2025](#)). I captured requirements in docs/requirements.md and tracked weekly milestones on GitHub. Likely risks such as weak security or database connection failure were logged in docs/risk-assessment.md early on.

Commercial platforms such as PagerDuty and ServiceNow already offer on-call routing, escalation policies, and integrations with cloud monitoring tools ([PagerDuty, 2024](#)). This project is not trying to replace those systems at enterprise scale. The B9IS109 brief required a smaller in-house register that still proves full-stack skills. MongoDB was a better fit than a relational database because each incident carries a changing timeline and comment thread. Embedding those arrays inside one document means the API can load a full incident history in a single query instead of joining several SQL tables. That kept the dashboard and detail page practical to build within the module timeframe ([MongoDB Inc., 2025](#); [Mongoose, 2025](#)).

1.2 Report Structure

This report documents my Cloud Incident Monitoring and Response Platform. The client uses HTML, CSS, and vanilla JavaScript modules ([Mozilla Developer Network, 2024b](#)). The server uses Node.js with Express ([Node.js, 2025](#); [Express.js, 2025](#)). Data is stored in MongoDB Atlas ([MongoDB Inc., 2025](#)). Real-time updates use Socket.IO ([Socket.IO, 2025](#)). Production deployment uses AWS EC2 with Nginx and PM2 ([Amazon Web Services, 2024](#); [Nginx, 2024](#); [PM2, 2024](#)). The sections cover overview, requirements, architecture, data model, CRUD, additional features, testing, external APIs, and conclusion. Source code: [Insert your public GitHub repository URL here].

2. Project Overview

2.1 Problem Statement

When cloud services fail or slow down, teams need one shared register for what broke, who is fixing it, and when service returned. Alerts may arrive from AWS CloudWatch, Azure Monitor, or Google Cloud Monitoring ([Amazon Web Services, 2024](#); [Microsoft, 2024](#); [Google, 2024](#)), but

the response work still needs a single ticket list. Without that, details get lost across chat threads and spreadsheets.

2.2 Full-Stack Objectives

My aim was a working end-to-end web application. On the frontend, users log in, open a dashboard, search incidents, create records, and change status in one interface. On the backend, Express routes validate input, check JWT tokens, and persist data in MongoDB ([Express.js, 2025](#); [JSON Web Token Introduction, 2024](#)). I also wanted Guest, User, and Administrator roles, webhook ingestion for external alerts, a layout that works on smaller screens, and a live cloud deployment.

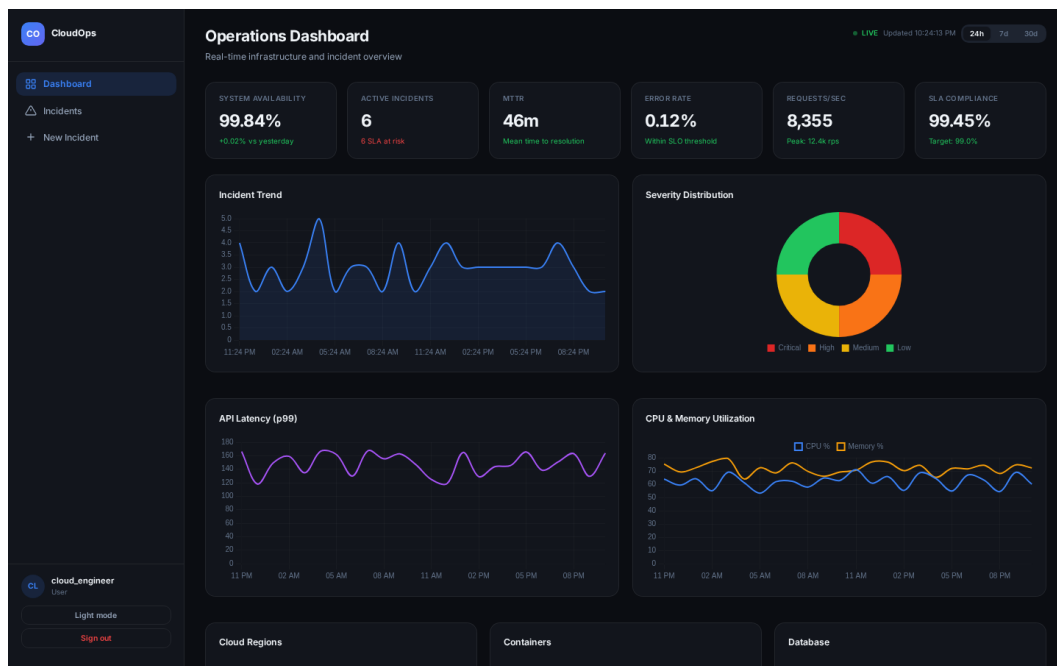


Figure 1: The Dashboard

3. System Requirements

3.1 Functional Requirements

The system was shaped around these functional needs. Users register and log in securely. Authorised users create incidents with title, description, severity, cloud provider, affected service, region, assigned team, and status. All incidents appear in a searchable table with severity and status filters plus pagination. Users edit records, update status and root cause, and post comments. Administrators delete incidents. Input is validated on the server before MongoDB writes ([express-validator, 2024](#)). The dashboard shows active counts, charts, and health panels

([Chart.js, 2025](#)). External monitoring tools submit alerts through a webhook. Each incident keeps a timeline of status changes and comments for audit.

3.2 Non-Functional Requirements

The server runs on Node.js with Express and stores data in MongoDB Atlas ([Node.js, 2025](#); [MongoDB Inc., 2025](#)). API responses use normal HTTP status codes such as 201 for create, 400 for validation failure, 401 for unauthorised access, and 404 when a record is missing ([Fielding, 2000](#)). The interface must work on desktop and smaller screens through responsive CSS. Security includes JWT login, role-based access, bcrypt password hashing ([bcryptjs, 2024](#); [OWASP Foundation, 2021](#)), rate limiting on auth routes ([express-rate-limit, 2024](#)), and Helmet security headers ([Helmet, 2025](#)). The app deploys to AWS EC2. Errors are handled in one server middleware module with logging for troubleshooting.

4. System Architecture

4.1 Client-Server Design

The application follows a client-server layout that is standard in web information systems ([Fielding, 2000](#)). The server uses Node.js and Express 5 and exposes REST endpoints under /api/auth, /api/incidents, /api/observability, and /api/cost. The frontend is one HTML page that loads JavaScript modules and calls the backend with the Fetch API ([Mozilla Developer Network, 2024a](#)). Socket.IO broadcasts create and update events so other open browser sessions refresh without a manual reload ([Socket.IO, 2025](#)).

This split matters in practice. The backend owns storage, validation, authentication, and business rules. The frontend shows data and captures user actions. Because every action goes through the API, the same backend could later support a mobile client without major rewrites.

4.2 Folder Structure

The server/ folder holds the entry point, routes, controllers, Mongoose models, and middleware. client/ contains the SPA HTML, CSS, and JavaScript modules. Shared enum values for severity, status, teams, and SLA minutes sit in server/shared-taxonomy.js so client and server stay aligned. deploy/ holds Nginx and PM2 scripts ([Nginx, 2024](#); [PM2, 2024](#)). Dependencies and npm scripts are listed in server/package.json. In production the path is: browser to Nginx on port 80, then PM2-managed Node.js on port 5000, then MongoDB Atlas.

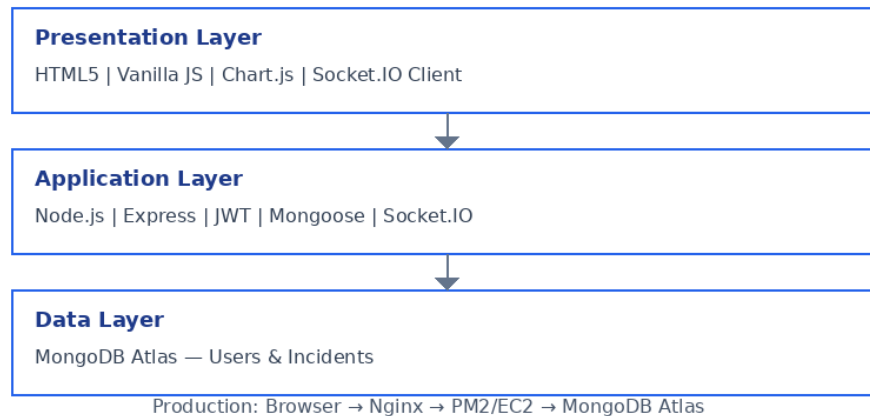


Figure 2: Architecture of the System

5. Data Model

5.1 Incident Document

Incidents are stored as MongoDB documents through Mongoose ([Mongoose, 2025](#); [MongoDB Inc., 2025](#)). Each record has an incidentId generated automatically in the form INC-YYYY-#####, for example INC-2026-0004. Required fields include title, description, severity (Critical, High, Medium, Low), status (Investigating, Identified, Monitoring, Resolved), cloudProvider (AWS, Azure, GCP, Other), affectedService, and region. assignedTeam and assignedToPerson route the work. detectedAt, slaDeadline, and resolvedAt track timing. A pre-save hook sets slaDeadline from severity: Critical allows sixty minutes, High four hours, Medium eight hours, and Low twenty-four hours.

Two nested arrays hold history. timeline stores detection events, status changes, and comment entries with author and timestamp. comments stores user updates separately. The source field records whether the incident was created manually, from a cloud webhook, or from monitoring. reportedBy links to the user who logged it. Indexes on status, severity, and createdAt speed up list queries; a text index supports search ([MongoDB Inc., 2025](#)).

5.2 User Document

Users have username, email, bcrypt-hashed password, and role. Roles are Guest, User, or Administrator. Passwords are hashed before save with ten salt rounds following common storage guidance ([OWASP Foundation, 2021](#); [bcryptjs, 2024](#)) and are never returned in API responses. Registration cannot grant Administrator directly; that role is seeded for demo accounts such as ops@platform.com.

6. CRUD Operations

The core of the system is CRUD on incidents through `/api/incidents`. Logic sits in `incidentController.js` and routes in `incidentRoutes.js`.

6.1 Create

Create is handled by `POST /api/incidents`. The user must be logged in with User or Administrator role. `express-validator` checks the body—title length, valid enums, and that the affected service matches the chosen cloud provider ([express-validator, 2024](#)). The controller saves a new document with source Manual, an opening timeline entry, and `reportedBy` taken from the JWT. On success the server returns `HTTP 201` and emits `incident:created` through `Socket.IO` ([Socket.IO, 2025](#)). Missing or invalid fields return `HTTP 400`.

6.2 Read

Read uses several `GET` endpoints. `GET /api/incidents` returns a paginated list. Query parameters support search text, status, severity, team, page, and limit. Search matches title, `incidentId`, and `affectedService` without case sensitivity. `GET /api/incidents/:id` returns one full record. `GET /api/incidents/metrics` returns counts and SLA breach figures for the dashboard. If the id does not exist the server returns `HTTP 404` ([Fielding, 2000](#)).

6.3 Update

Update uses `PUT /api/incidents/:id`. Only fields sent in the body are changed; others stay as they were. When status changes, a new timeline entry is added. If status becomes Resolved, `resolvedAt` is set. Comments are added through `POST /api/incidents/:id/comments`, which also writes to the timeline. A successful update returns `HTTP 200` and broadcasts `incident:updated` through `Socket.IO` ([Socket.IO, 2025](#)).

6.4 Delete

Delete uses `DELETE /api/incidents/:id` and is limited to Administrator. The handler checks the record exists, removes it, returns `HTTP 200`, and emits `incident:deleted`. Unknown ids return `HTTP 404`.

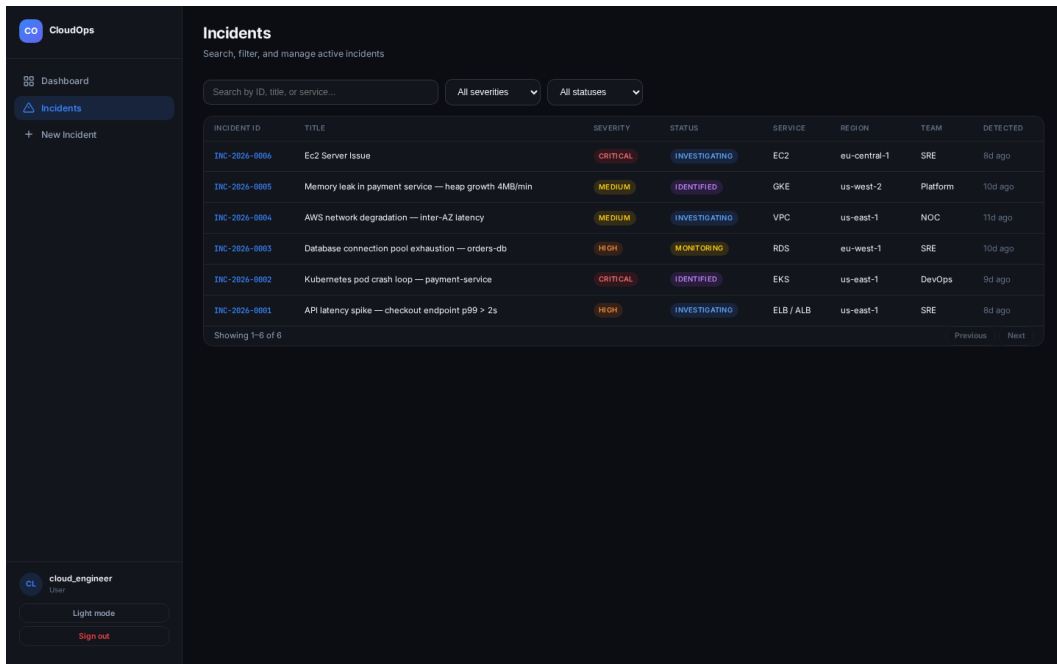


Figure 3: Searching of Incidents with Edit and Delete

The screenshot shows the 'Create Incident' form in CloudOps. The left sidebar is the same as in Figure 3. The main area is titled 'Create Incident' with a subtitle 'Report a new infrastructure or service incident'. The form contains the following fields:

- Incident Title:** A text input field with a placeholder 'Brief description of the incident'.
- Severity:** A dropdown menu with 'Select severity'.
- Status:** A dropdown menu with 'Select status'.
- Issue Type:** A dropdown menu with 'Select issue type'.
- Region:** A dropdown menu with 'Select region'.
- Cloud Provider:** A dropdown menu with 'Select provider'.
- Service Impacted:** A dropdown menu with 'Select service'.
- Assigned Team:** A dropdown menu with 'Select team'.
- Owner:** A text input field with a placeholder 'Engineer name'.
- Description:** A text area with a placeholder 'Impact, symptoms, and initial findings'.
- Root Cause (if known):** A text area with a placeholder 'Optional — document if already identified'.

At the bottom of the form is a blue 'Create Incident' button. The bottom sidebar shows the user 'cloud_engineer' with 'Light mode' and 'Sign out' buttons.

Figure 4: Form for Add and Edit Incident

7. Additional Features

Search takes a text query and compares it against title, incidentId, and affectedService. Severity and status dropdowns narrow the list further. Pagination handles larger datasets. Input on the search box is debounced so the API is not called on every keystroke.

The dashboard shows KPI cards for availability, active incidents, mean time to resolve, and error rate. Chart.js draws graphs for incident trends, API latency, CPU, and memory ([Chart.js, 2025](#)). Values refresh every thirty seconds and also when Socket.IO events arrive. Severity badges use colour coding so Critical rows stand out at a glance.

The incident detail page shows an SLA countdown, root cause field, and full timeline. JWT authentication and role-based access control gate what each user can do ([JSON Web Token Introduction, 2024](#)). Guests can view only; Users create and update; Administrators can delete. Helmet, rate limits, and input validation run on the server ([Helmet, 2025](#); [express-rate-limit, 2024](#)). CSS media queries at 1024px and 900px collapse the sidebar and stack columns on smaller screens. Theme toggle switches between dark and light mode.

8. Testing and Validation

Testing was done manually during development and before deployment. I checked login with valid and invalid credentials. Create, read, update, and delete paths were exercised through the web UI and with direct HTTP requests. Search and filter combinations were tried to confirm the correct rows returned. RBAC was tested by logging in as Guest, User, and Administrator and confirming forbidden actions returned HTTP 403. The webhook endpoint was tested with curl using the x-webhook-secret header. The /api/health endpoint confirmed database connectivity on EC2 ([Amazon Web Services, 2024](#)).

Validation errors were checked by submitting empty required fields and confirming HTTP 400 responses ([express-validator, 2024](#)). Two browser windows were opened to confirm Socket.IO updated the incident list when one session created a record ([Socket.IO, 2025](#)). Automated Jest and Supertest suites are not yet part of this codebase but would be a sensible next step for regression testing.

Manual API Test Results			
Cloud Incident Monitoring Platform — B9IS109			
Endpoint	HTTP	Result	Notes
GET /api/health	200	PASS	{'status': 'healthy', 'db': 'connected', 'uptime
POST /api/auth/login	200	PASS	JWT token received
GET /api/incidents	200	PASS	5 incidents returned
GET /api/incidents/metrics	200	PASS	Metrics OK
GET /api/observability/dashboard	200	PASS	KPI data OK
GET /api/incidents (no token)	401	PASS	401 expected
Summary: 6/6 tests passed			

Figure 5: Manual Test Results

9. External API Integration

The system consumes and exposes web services using JSON throughout ([Fielding, 2000](#)). The frontend api.js module calls internal REST endpoints for auth, incidents, metrics, observability, and cost data ([Mozilla Developer Network, 2024a](#)).

For external integration, the cost controller supports an optional `COST_API_URL` environment variable. When set, the server fetches billing or utilisation data from that remote service and forwards the JSON response to the client. If the remote call fails, the handler logs a warning and returns structured simulated multi-cloud figures so the demo still works offline.

Inbound webhook integration allows CloudWatch, Azure Monitor, or similar tools to POST alerts to `/api/incidents/webhook` ([Amazon Web Services, 2024](#); [Microsoft, 2024](#); [Google, 2024](#)). These systems cannot use JWT login, so they authenticate with a shared secret header instead. Valid payloads create incidents with source `CloudWebhook` and appear on the dashboard through Socket.IO ([Socket.IO, 2025](#)). This matches how alert forwarders often connect to ticketing tools in production.

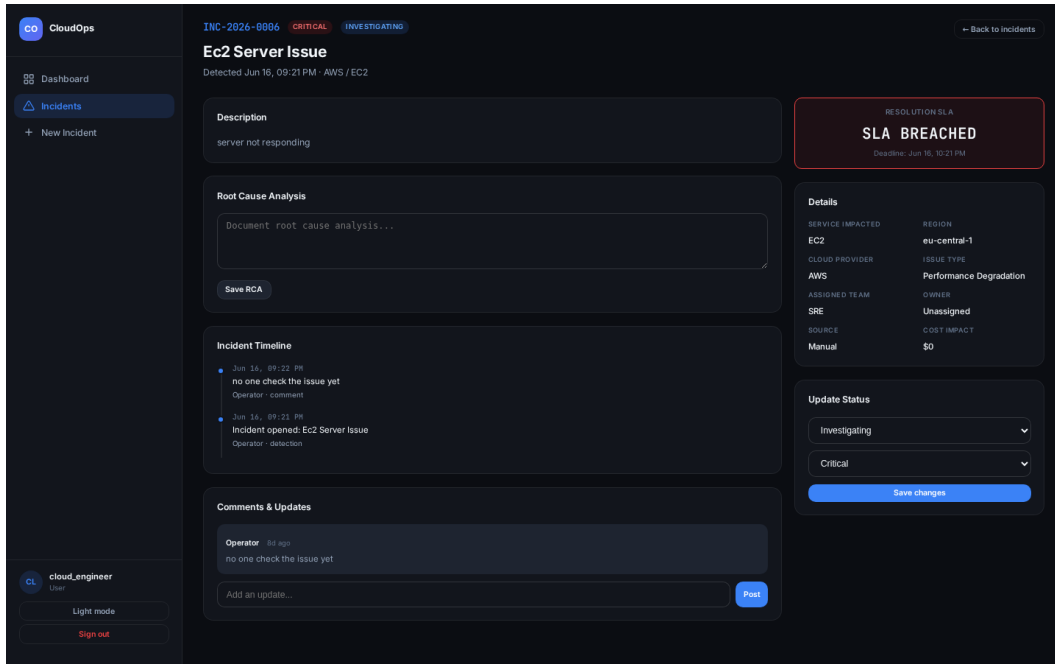


Figure 6: Incident Detail and Timeline

10. Conclusion

The Cloud Incident Monitoring and Response Platform meets the requirements set out in the B9IS109 brief. A REST API provides full CRUD on incidents ([Fielding, 2000](#)). The web interface talks to the backend only through API and WebSocket calls ([Socket.IO, 2025](#)). Search, dashboards, SLA tracking, real-time sync, webhook ingestion, JWT authentication, role-based access, responsive layout, and AWS EC2 deployment are all in place.

The application is built in JavaScript throughout: Node.js and Express on the server ([Node.js, 2025](#); [Express.js, 2025](#)), HTML and ES modules on the client ([Mozilla Developer Network, 2024b](#)), with MongoDB Atlas for storage ([MongoDB Inc., 2025](#)). The code is split into routes, controllers, models, and client modules, and third-party libraries such as Chart.js and Socket.IO are listed in package.json ([Chart.js, 2025](#); [Socket.IO, 2025](#)).

The project demonstrates a complete full-stack web information system: frontend, API, database, security, real-time communication, and cloud deployment working together as one deliverable.

Reflective Learning Experience

Working on this project helped me join separate coding tasks into one working system. Tracing a login from the form through JWT middleware to a MongoDB query made the three-tier layout much clearer ([JSON Web Token Introduction, 2024](#); [MongoDB Inc., 2025](#)). Socket.IO was new to me—I had to emit events only after a successful save and refresh the open client view

([Socket.IO, 2025](#)). Deploying to EC2 exposed issues localhost hides, such as opening port 80, whitelisting the server IP in MongoDB Atlas, and setting environment variables correctly ([Amazon Web Services, 2024](#); [MongoDB Inc., 2025](#)). I also saw why server-side role checks matter even when the UI hides buttons. If I started again I would add automated tests earlier. Overall it was a solid first attempt at building and hosting a web app for cloud operations.

References

Amazon Web Services (AWS) (2024) *Amazon CloudWatch Documentation*. Available at: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/>.

Amazon Web Services (AWS) (2024) *Amazon EC2 Documentation*. Available at: <https://docs.aws.amazon.com/ec2/>.

bcryptjs (2024) *bcryptjs Documentation*. Available at: <https://www.npmjs.com/package/bcryptjs>.

Chart.js (2025) *Chart.js Documentation*. Available at: <https://www.chartjs.org/docs/latest/>.

Crockford, D. (2008) *JavaScript: The Good Parts*. Sebastopol: O'Reilly Media.

Express.js (2025) *Express Documentation*. Available at: <https://expressjs.com/>.

express-rate-limit (2024) *express-rate-limit Documentation*. Available at: <https://express-rate-limit.mintlify.app/>.

express-validator (2024) *express-validator Documentation*. Available at: <https://express-validator.github.io/docs/>.

Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation. University of California, Irvine. Available at: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>.

Google (2024) *Google Cloud Monitoring Documentation*. Available at: <https://cloud.google.com/monitoring/docs>.

Helmet (2025) *Helmet.js Documentation*. Available at: <https://helmetjs.github.io/>.

JSON Web Token Introduction (2024) *Introduction to JSON Web Tokens*. Available at: <https://jwt.io/introduction>.

Microsoft (2024) *Azure Monitor Documentation*. Available at: <https://learn.microsoft.com/en-us/azure/azure-monitor/>.

MongoDB Inc. (2025) *MongoDB Atlas Documentation*. Available at: <https://www.mongodb.com/docs/atlas/>.

Mongoose (2025) *Mongoose Documentation*. Available at: <https://mongoosejs.com/docs/>.

Mozilla Developer Network (2024a) *Fetch API Documentation*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.

Mozilla Developer Network (2024b) *JavaScript Modules Documentation*. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>.

Node.js (2025) *Node.js Documentation*. Available at: <https://nodejs.org/docs/latest/api/>.

Nginx (2024) *NGINX Documentation*. Available at: <https://nginx.org/en/docs/>.

OWASP Foundation (2021) *Password Storage Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html.

PagerDuty (2024) *PagerDuty Documentation*. Available at: <https://developer.pagerduty.com/>.

PM2 (2024) *PM2 Documentation*. Available at: <https://pm2.keymetrics.io/docs/usage/quick-start/>.

Socket.IO (2025) *Socket.IO Documentation*. Available at: <https://socket.io/docs/v4/>.